



Universidad
Nacional de
General
Sarmiento

Instituto de Industria
Licenciatura en Sistemas

TRABAJO FINAL

Sistemas Operativos y Redes 2

Primer Semestre 2026

"Análisis de registros de auditoría con LLM local: Evaluación de phi3:mini frente a herramientas nativas"

Alumno/a	Sturtz Tomás
Legajo	39761866/2016
Docente	Benjamin Chuquimango
Fecha de entrega	19/06/2026

Modalidad Individual — Obligatorio

Buenos Aires, Argentina — 2026

Índice de contenidos

1. Resumen ejecutivo.....	3
2. Planteamiento del problema.....	4
3. Pregunta de investigación y objetivos.....	6
3.1 Pregunta de investigación.....	6
3.2 Objetivo general.....	6
3.3 Objetivos específicos.....	6
4. Marco teórico.....	7
4.1 Auditoría del sistema operativo y logs.....	7
4.2 Large Language Models (LLMs).....	7
4.3 LLMs aplicados a la seguridad de la información y privacidad de datos.....	8
4.4 Clasificación de eventos y fatiga por alertas.....	8
4.5 Few-shot e ingeniería de prompts.....	9
4.6 Evaluación de modelos pequeños en tareas técnicas.....	9
5. Metodología.....	11
5.1 Enfoque metodológico.....	11
5.2 Herramientas y justificación.....	12
6. Implementación y desarrollo.....	14
6.1 Diseño de la solución.....	14
6.2 Implementación.....	15
6.2.1 Generación y captura de eventos.....	15
6.2.2 Baseline.....	17
6.2.3 Optimización de los datasets.....	18
6.2.4 Modelos y motor de inferencia.....	18
6.2.5 Inferencia y prompts.....	18
6.3 Análisis según modelo OSI o capas del SO.....	19
6.4 Pruebas y verificación.....	20
6.4.1 Marco de evaluación y Ground Truth.....	20
6.4.2 LLM vs Baseline.....	21
6.4.3 Pruebas de formatos.....	21
6.4.4 Pruebas de prompts.....	22
6.4.5 Pruebas de modelos.....	22
6.4.6 Pruebas de reproducibilidad y significancia estadística.....	22
6.4.7 Pruebas de logs de ruido.....	23
7. Resultados y análisis.....	24
7.1 Comparación con los objetivos.....	33
7.2 Limitaciones identificadas.....	34
7.3 Propuestas de mejora y trabajo futuro.....	34
8. Conclusiones.....	36
9. Referencias bibliográficas.....	38
10. Anexos.....	39
Anexo A — Prompts.....	39
Anexo B — Código de la solución.....	40

1. Resumen ejecutivo

Este trabajo aborda la problemática de la sobrecarga operativa y la fatiga por alertas que sufren los administradores de sistemas al analizar manualmente los miles de registros crudos generados por el subsistema nativo de auditoría en sistemas operativos Linux. Frente al riesgo de cumplimiento de privacidad asociado al envío de logs hacia servidores externos en la nube, se propone determinar la viabilidad de desplegar un pipeline automatizado local para interpretar dichos registros. La metodología implementada posee un enfoque experimental en laboratorio, simulando cinco escenarios de vulnerabilidad y actividad rutinaria de ruido mediante una máquina virtual. Se diseñó un orquestador en Python que procesa unitariamente los eventos mediante aprendizaje en contexto y el motor de inferencia Ollama, contrastando modelos de lenguaje compactos en hardware limitado a cuatro gigabytes de memoria de video. Los resultados demuestran que el modelo alcanza una precisión superior al 70% operando sobre el formato de texto crudo original. No obstante, formatos compactos estructurados como JavaScript Object Notation degradan notablemente el razonamiento semántico del modelo, e introducen una latencia fija promedio de 6.69 segundos por registro que limita su uso en tiempo real. La contribución del trabajo consiste en validar que un modelo de lenguaje local puede actuar como una segunda capa de asistencia interpretativa y traducción a lenguaje natural, complementando el filtrado determinista clásico sin comprometer la privacidad de los datos internos.

Palabras clave: Análisis de logs, Modelos de lenguaje locales, Seguridad en linux, auditd, Ingeniería de prompts

2. Planteamiento del problema

En el ámbito de la seguridad y la administración de sistemas operativos basados en GNU/Linux, la auditoría del sistema es un pilar fundamental para garantizar la integridad, la confidencialidad y la disponibilidad de los recursos organizacionales. La herramienta nativa estándar para este propósito es el subsistema *auditd*, el cual permite registrar de manera detallada eventos de seguridad a nivel de llamadas al sistema (*syscalls*). Sin embargo, la efectividad de este mecanismo se ve limitada por la gran cantidad de datos generados en entornos de producción reales.

El problema radica en que un administrador de sistemas promedio recibe cientos o incluso miles de eventos individuales provenientes de *auditd* por día. El proceso tradicional de auditoría requiere que dicho administrador revise estos registros línea por línea de manera manual para identificar comportamientos sospechosos o maliciosos. Este enfoque consume un tiempo operativo excesivo y se convierte en un cuello de botella para la respuesta temprana ante incidentes. La fatiga por alertas y la imposibilidad material de procesar e interpretar tal volumen de información de forma oportuna provocan que eventos anómalos o de alto riesgo pasen desapercibidos, elevando exponencialmente el riesgo tecnológico de la organización (Cadet et al., 2024).

Para mitigar esta problemática, se emplean herramientas clásicas de análisis y filtrado como *ausearch* y *aureport*. Si bien estas utilidades cumplen eficazmente con la función de aislar registros basados en criterios estáticos predefinidos, son herramientas puramente sintácticas. Esto significa que no pueden interpretar el significado técnico del log ni evaluar de forma autónoma el nivel de riesgo que representa cada evento individual. El administrador sigue estando obligado a leer los registros en formato RAW, comprender qué implicancia tiene la *syscall* ejecutada y determinar el peligro real de la acción de forma completamente manual, lo cual vuelve inviable la escalabilidad del monitoreo.

Ante la necesidad de automatizar la interpretación y clasificación del texto estructurado de los logs, los LLMs, emergen como una alternativa con capacidades avanzadas de razonamiento interpretativo. Un pipeline automatizado que procese los eventos de manera individual puede clasificar el nivel de riesgo en segundos, traduciendo la salida técnica de una llamada al sistema a un resumen explicativo en lenguaje natural que el administrador puede evaluar en poco tiempo.

No obstante, la implementación convencional de estas tecnologías mediante APIs comerciales en la nube introduce un severo conflicto de cumplimiento normativo y seguridad. Enviar logs del sistema operativo —los cuales contienen metadatos críticos, variables de entorno, nombres de usuarios reales e información interna de la infraestructura— hacia servidores externos de terceros vulnera la privacidad de los datos, eleva el riesgo de filtración personal (Das et al., 2024) y expone la superficie de ataque de la organización.

Por lo tanto, el verdadero desafío técnico y metodológico radica en determinar la viabilidad de desplegar un pipeline automatizado basado en un LLM de ejecución estrictamente local (utilizando motores de inferencia optimizados como *Ollama*). Este enfoque promete resolver el problema de la sobrecarga de datos y el tiempo de análisis manual mediante la clasificación eficiente de eventos individuales, manteniendo el control absoluto de la información dentro de la seguridad del propio sistema operativo, sin depender de conectividad a Internet ni comprometer la privacidad.

3. Pregunta de investigación y objetivos

3.1 Pregunta de investigación

¿Puede el modelo *phi3:mini* utilizando *few-shot prompting* detectar eventos de seguridad en logs de *auditd* con mayor precisión que reglas manuales de *ausearch*, en hardware con GPU de 4GB sin conexión a servicios externos?

3.2 Objetivo general

Evaluar la viabilidad técnica y la precisión del modelo de lenguaje de ejecución local *phi3:mini* utilizando *few-shot prompting* para la clasificación automatizada y la interpretación del nivel de riesgo de eventos de auditoría de *auditd*, contrastando cuantitativamente su rendimiento frente a las utilidades nativas de filtrado del sistema operativo bajo condiciones estrictas de un hardware con GPU de 4GB.

3.3 Objetivos específicos

1. Contrastar los resultados de cada escenario del LLM contra el baseline en términos de precisión, coincidencia y tiempo promedio de respuesta.
2. Evaluar la influencia de la estructura de los datos en la capacidad de comprensión del modelo, comparando la tasa de precisión del LLM y el tiempo promedio de respuesta al procesar tres formatos distintos: RAW completo, RAW reducido y JSON reducido.
3. Evaluar y analizar la variación en el porcentaje de aciertos del modelo ante diferentes instrucciones y ejemplos provistos.
4. Evaluar el rendimiento de los modelos *phi3:mini* y *llama3.2:3b* en cuanto a la precisión y tiempo promedio de respuesta.

4. Marco teórico

4.1 Auditoría del sistema operativo y logs

En sistemas operativos basados en GNU/Linux, la visibilidad sobre las acciones del espacio de usuario depende de la capacidad del sistema para interceptar e indexar las llamadas al sistema (*syscalls*). Las *syscalls* representan la interfaz fundamental a través de la cual los procesos solicitan recursos o privilegios al núcleo (*kernel*). Para registrar estos eventos de manera segura e inmutable, el núcleo dispone del subsistema nativo *auditd*, el cual opera interceptando de forma directa el flujo de las llamadas al sistema mediante reglas preconfiguradas.

Los eventos capturados por *auditd* se estructuran en registros semiestructurados clave-valor que detallan metadatos críticos como el identificador del usuario real (*uid*), el usuario efectivo (*euid*), el comando (*comm*), entre otros. Herramientas clásicas de filtrado como *ausearch* y *aureport* actúan de forma determinista sobre estos registros, aislando líneas de texto mediante coincidencias exactas o llaves de auditoría específicas.

En lo que respecta al análisis y la interpretación de estos registros para la detección de comportamientos anómalos, el orden y la densidad informativa de los logs limitan drásticamente a las herramientas automáticas tradicionales. Karlsen et al. (2023) exponen que los enfoques basados en análisis sintáctico y configuraciones estáticas son inflexibles ante variaciones en la estructura de los datos, lo que motiva el uso de modelos de lenguaje para extraer características de forma semántica y sin procesamiento previo. No obstante, la investigación se restringe a registros genéricos de aplicaciones web y *syslogs* estándar en el espacio de usuario, omitiendo por completo el comportamiento de bajo nivel de las llamadas al sistema recolectadas por *auditd*, donde la pérdida de un solo campo técnico puede inhabilitar la comprensión del contexto operativo del evento.

4.2 Large Language Models (LLMs)

Los LLMs son modelos de aprendizaje profundo avanzados que se caracterizan por contener arquitecturas de miles de millones de parámetros entrenadas sobre una cantidad enorme de datos textuales. Esta escala masiva de parámetros permite al sistema adquirir una capacidad de comprensión general del lenguaje y de generación de contenido continuo, permitiéndole internalizar complejas estructuras morfosintácticas y semánticas a partir de datos no estructurados (Minaee et al., 2024).

A nivel de diseño computacional, los LLMs contemporáneos se estructuran fundamentalmente sobre la arquitectura de redes neuronales conocida como *Transformer*, la cual aporta una infraestructura de procesamiento basada en mecanismos de atención que destaca por su eficiencia para capturar patrones lingüísticos y dependencias secuenciales complejas en el texto, superando las limitaciones de pérdida de contexto analítico que afectaban a los modelos recurrentes tradicionales (Zhao et al., 2023; Minaee et al., 2024).

Los LLMs generativos actúan como modelos predictivos de carácter autorregresivo. Su funcionamiento mecánico se basa en la descomposición de un texto de entrada (*prompt*) en unidades mínimas de información denominadas *tokens*; a partir de esta secuencia inicial, el modelo ejecuta un cálculo probabilístico iterativo para predecir y generar el *token* subsiguiente más adecuado según el contexto analizado (Minaee et al., 2024). Este principio predictivo es el que dota a estos modelos de la capacidad

semántica para interpretar cadenas técnicas de texto semiestructurado y traducirlas a descripciones en lenguaje natural.

4.3 LLMs aplicados a la seguridad de la información y privacidad de datos

Frente a las limitaciones de los analizadores tradicionales basados puramente en expresiones regulares, la evolución de las arquitecturas de aprendizaje profundo ha permitido aplicar el Procesamiento de Lenguaje Natural (PLN) a la auditoría técnica. Las capacidades cognitivas y de razonamiento contextual de los LLMs superan a las herramientas estáticas en la interpretación de alertas técnicas y su correlación con políticas de seguridad organizacionales, reduciendo de forma medible el tiempo medio de detección (Cadet et al., 2024). Sin embargo este estudio orienta su enfoque de manera casi exclusiva hacia el despliegue de modelos comerciales en infraestructuras de nube pública. Al depender de APIs de terceros, omiten el análisis del rendimiento y la degradación de precisión que sufren los modelos más pequeños cuando operan bajo restricciones de hardware local, dejando un vacío sobre la viabilidad real de procesar auditorías técnicas de bajo nivel sin depender de APIs externas.

La necesidad de restringir el flujo de datos técnicos hacia el exterior es crítica. Procesar logs corporativos —los cuales contienen nombres de usuarios reales, variables de entorno y rutas internas del sistema operativo— a través de servicios en la nube vulnera las normativas de cumplimiento de privacidad y expone a la organización a filtraciones de información personal y ataques a su superficie (Das et al., 2024). Frente a esta problemática evidenciada, mi propuesta válida la utilización local y aislada de los LLMs mediante motores como Ollama como la alternativa técnica más segura. Sin embargo, estudios como el de Das et al. (2024) se centran en el mapeo de vulnerabilidades y omiten estrategias de optimización que permitan mitigar la pérdida de capacidad de razonamiento que experimentan los modelos de lenguaje al ser reducidos en escala para ajustarse a plataformas locales de cómputo limitado.

La automatización en el análisis de alertas masivas mediante diferentes estrategias de interacción permite clasificar amenazas de forma eficiente. Tejero-Fernández y Sánchez-Macián (2025) evalúan el rendimiento de modelos de código abierto utilizando técnicas de few-shot prompting y ajuste fino (fine-tuning) para categorizar riesgos de ciberseguridad. En su estudio, demuestran que los LLMs sometidos a un ajuste fino superan a los algoritmos tradicionales de *Machine Learning* en la clasificación multiclase de ataques. Además, los autores logran que los modelos no solo clasifiquen el tipo de amenaza, sino que generen simultáneamente acciones de mitigación adaptadas al entorno, obteniendo una alta precisión en sus recomendaciones. Sin embargo, su experimentación se limita a registros estructurados de tráfico de red en entornos específicos de conectividad. Esto deja fuera el comportamiento de las llamadas al sistema de bajo nivel y no analiza cómo la optimización o reducción de campos altera la ventana de atención de un modelo compacto local.

4.4 Clasificación de eventos y fatiga por alertas

El procesamiento automatizado de logs no solo busca detectar amenazas latentes, sino resolver el problema operacional de la saturación de datos que sufren los administradores de sistemas ante la recepción diaria de miles de eventos rutinarios (ruido del sistema). La capacidad de un clasificador para asignar niveles de riesgo diferenciados (como ALTO, MEDIO, BAJO e INFO) permite jerarquizar la atención del administrador de manera eficiente.

Sobre esto, Kheddar (2024) examina en su revisión el rol de los modelos basados en Transformers implementados en sistemas de detección de intrusos. El autor recopila evidencia en la literatura reciente sobre cómo la integración de LLMs disminuye de manera drástica la fatiga por alertas al traducir la sintaxis técnica de los registros de auditoría a resúmenes descriptivos legibles en lenguaje natural. Sin embargo, como el propio Kheddar advierte, el despliegue de estos modelos de lenguaje enfrenta severos desafíos de escalabilidad relacionados con la demanda de procesamiento en tiempo real. La complejidad de estas arquitecturas asume capacidades de cómputo de alto rendimiento, lo que representa un cuello de botella crítico y una barrera de viabilidad para la realidad de los entornos locales restringidos en memoria (como la VRAM), una limitación técnica que la presente investigación busca mitigar.

4.5 Few-shot e ingeniería de prompts

Para lograr que un modelo de lenguaje adquiriera la competencia técnica necesaria para interpretar la sintaxis cruda de una llamada al sistema se recurre al paradigma del aprendizaje en contexto (*In-Context Learning*), en el cual al modelo se le da una instrucción (prompt) que incluye un conjunto de ejemplos como parte de la secuencia de entrada. El aprendizaje en contexto funciona reconociendo la relación entre los ejemplos de la instrucción y aplicando la misma correlación a las nuevas entradas (Varughese, s.f.).

Dentro de estas estrategias de interacción, el prompting *few-shot* se fundamenta en la inclusión explícita de pares estructurados de ejemplos de entrada y salida correctas previos a la consulta real. Esto permite condicionar el mecanismo de atención del modelo, enseñándole a reconocer patrones de sintaxis específicos y a estructurar su respuesta de forma determinista bajo un formato predefinido, permitiendo al modelo realizar tareas con un mínimo de ejemplos (Gadesha, s.f.).

Al evaluar la especialización operativa de modelos mediante instrucciones dirigidas, Xu et al. (2024) aportan una revisión sistemática enfocada en el uso de LLMs para ciberseguridad. En su estudio, identifican cómo el aprendizaje en contexto y la ingeniería de prompts se han consolidado como estrategias altamente efectivas para la clasificación técnica, evadiendo la necesidad de infraestructuras masivas de cómputo vinculadas al reentrenamiento de modelos. Sin embargo, la literatura recopilada en dicha revisión omite la evaluación de modelos pequeños (de menos de 4.000 millones de parámetros), donde la sobrecarga de ejemplos extensos en el prompt consume rápidamente los tokens de atención disponibles, incrementando la propensión del modelo a sufrir colapsos en el razonamiento técnico o alucinaciones semánticas graves.

4.6 Evaluación de modelos pequeños en tareas técnicas

El despliegue de modelos reducidos en entornos con recursos limitados representa una alternativa eficiente y económica frente a los grandes modelos comerciales de la nube. Belcak et al. (2025) defienden el uso de modelos de lenguaje pequeños (SLMs) para ejecutar tareas técnicas especializadas y repetitivas, evaluando su capacidad en la generación automática de código y en la optimización de flujos de trabajo sobre interfaces de software. En su trabajo, los autores aportan un algoritmo metodológico de conversión diseñado para migrar ecosistemas dependientes de LLMs hacia arquitecturas basadas en SLMs. Mediante el análisis de casos de estudio sobre agentes autónomos de código abierto, demuestran que entre el 40% y el 70% de las invocaciones a modelos masivos pueden ser reemplazadas confiablemente por modelos especializados locales. Además, ratifican que los SLMs contemporáneos

logran igualar el rendimiento de los modelos comerciales masivos en el uso de herramientas y seguimiento de instrucciones, reduciendo los costos de inferencia y latencia entre 10 y 30 veces.

A pesar de validar la viabilidad y el resguardo de la privacidad que ofrecen estos modelos compactos, el estudio se mantiene en un plano principalmente conceptual y de diseño de sistemas. En consecuencia, omite una validación empírica en el ámbito de la ciberseguridad, ignorando los problemas de saturación de contexto o alucinaciones semánticas que sufren estos modelos al procesar texto crudo de auditoría del sistema operativo.

Bui et al. (2024) realizan una comparación entre modelos de lenguaje de escala reducida y algoritmos clásicos de aprendizaje automático para la detección de intrusiones en auditorías de seguridad. En su estudio, aportan evidencia de que los LLMs compactos (de aproximadamente 110 millones de parámetros) sometidos a un ajuste fino superan consistentemente en precisión a las herramientas tradicionales de Machine Learning. Además, el artículo demuestra que utilizar modelos masivos para tareas de clasificación resulta ser un gasto innecesario de recursos técnicos (*overkill*) que no mejora la tasa de detección. Sin embargo, el estudio concentra su análisis estrictamente en eventos de tráfico de red, omitiendo por completo el análisis de logs del kernel.

Conocer el nivel de certeza de los modelos compactos permite saber hasta dónde pueden interpretar los datos de forma confiable antes de cometer errores. En esta línea, Ma et al. (2026) evalúan el uso de arquitecturas de escala reducida para tareas críticas, como la clasificación y la predicción del nivel de riesgo en los registros del sistema. Los autores proponen un mecanismo de evaluación para medir la incertidumbre del modelo local, demostrando que los registros mal clasificados coinciden casi siempre con altos índices de duda interna del propio sistema. A nivel operativo, utilizar este nivel de confianza como filtro inicial aportó resultados muy favorables, el modelo compacto logró resolver por sí solo la gran mayoría de los eventos rutinarios. Al derivar hacia un modelo masivo únicamente la fracción de casos más complejos, el entorno de trabajo demostró una reducción drástica en la sobrecarga computacional y en los costos de análisis, manteniendo una precisión comparable a la de los sistemas más avanzados.

Sin embargo, esta metodología requiere una fase de preparación. Obliga a que el modelo pequeño sea reentrenado (*fine-tuning*) de manera exhaustiva utilizando grandes volúmenes de datos previamente etiquetados y adaptados a cada sistema. Al depender de esta fase de entrenamiento, la investigación no evalúa la capacidad de los modelos pequeños para operar de forma directa e independiente usando únicamente instrucciones en contexto.

5. Metodología

5.1 Enfoque metodológico

El presente trabajo se rige bajo un enfoque metodológico de carácter experimental y empírico, desarrollado en un entorno controlado. Este diseño permite la manipulación y evaluación de variables independientes —tales como la arquitectura del modelo de lenguaje, el formato sintáctico del log de entrada y la estructura de la ingeniería de prompts— sobre variables dependientes medibles de manera cuantitativa, específicamente la precisión de la clasificación de riesgo y el tiempo promedio de inferencia por evento evaluado.

Para llevar a cabo la ejecución, se configuró un entorno de trabajo híbrido. La generación de los registros se realiza en una máquina virtual de Oracle VirtualBox que ejecuta el sistema operativo Linux Ubuntu Server 22.04 LTS. En este entorno virtualizado se generaron los registros de cinco eventos de *auditd* distintos. El escenario 1 contempla el acceso y lectura de archivos críticos del sistema (*/etc/passwd*), el escenario 2 simula intentos de escalada de privilegios ejecutados desde un usuario sin privilegios, el escenario 3 aborda la ejecución de comandos como usuario root, mientras que los escenarios 4 y 5 tienen que ver con la modificación de */etc/sudoers* y el acceso a */etc/shadow* por proceso no autorizado respectivamente. Todo esto se complementa con un dataset de actividad normal (ruido) diseñado específicamente para medir la tasa de falsos positivos.

Dentro de esta planificación, el Escenario 3 presentó inicialmente un volumen considerable de 652 eventos. Debido a la naturaleza de las operaciones de gestión de paquetes (*apt-get update*) y administración de servicios del sistema operativo, el subsistema de auditoría genera una enorme cantidad de registros repetitivos y redundantes que constituyen ruido. Esto generaría un incremento innecesario en el tiempo total de ejecución del experimento sin aportar valor analítico adicional ya que se cuenta con un dataset de ruido específicamente. Por consiguiente, se tomó la decisión de acotar este escenario mediante un filtrado enfocado en aislar únicamente los eventos vinculados al identificador del usuario real y a las acciones principales ejecutadas deliberadamente. Esta decisión optimiza la eficiencia y garantiza una evaluación limpia y directa de la capacidad de comprensión semántica de los modelos frente a las acciones objetivo.

Una vez finalizadas las pruebas de seguridad, estos archivos de logs son exportados hacia el sistema operativo anfitrión (*host*), el cual opera bajo el sistema operativo Windows. En este entorno se centraliza la ejecución del pipeline automatizado en Python y el despliegue del motor de inferencia local *Ollama*.

Para el desarrollo del trabajo se decidió utilizar como modelo principal *phi3:mini* (3.8B de parámetros) y realizar una evaluación comparativa con el modelo *llama3.2:3b* (3B de parámetros). La elección de estos dos modelos responde a la restricción física del hardware disponible, el cual cuenta con una GPU de 4GB de VRAM.

Respecto a las decisiones de diseño del pipeline, se descartó el enfoque de instrucciones directas (*zero-shot*) debido a la alta propensión a la alucinación técnica que exhiben los modelos más compactos. En su lugar, se adopta una estrategia de *few-shot prompting*. Este método permite condicionar los mecanismos de atención del modelo a través de la inclusión de pares estructurados de ejemplos de entrada y salida correctas previos a la consulta, garantizando la consistencia del formato de respuesta y la precisión interpretativa.

Asimismo, la granularidad del análisis se define a nivel unitario (log por log). Aunque el agrupamiento de registros en ventanas temporales facilitaría teóricamente la correlación secuencial de eventos, en la práctica satura de forma inmediata la ventana de contexto de los modelos más pequeños y dispara el consumo de VRAM, provocando fallos por desbordamiento de memoria. El procesamiento secuencial e individualizado garantiza la viabilidad operativa y la estabilidad del hardware bajo restricciones físicas.

Como contraparte analítica para la validación de los resultados, se estableció un *baseline* basado en la definición de reglas de búsqueda manuales mediante la utilidad nativa *ausearch*. Se evaluará de forma comparativa cuántos de los cinco escenarios de seguridad son detectados correctamente por el filtrado tradicional y por el LLM.

Además, se realizará una evaluación bajo tres estructuras sintácticas diferentes: RAW completo (formato por defecto de *ausearch*), RAW reducido y JSON reducido, los cuales serán contruidos a partir los registros del RAW completo a través de un script en Python. Se tiene como objetivo evaluar cómo el formato, la densidad de la información y la eliminación de metadatos redundantes del núcleo del sistema operativo alteran la capacidad del modelo para distinguir riesgos y optimizar sus tiempos de respuesta.

Luego con el formato que obtenga mejores resultados se probarán distintas variaciones del prompt para determinar cual entrega mejores resultados. Sabiendo que formato y que prompt obtiene mejores resultados, se hará la comparación con el modelo *llama3.2:3b* para determinar qué modelo resuelve mejor nuestro problema.

5.2 Herramientas y justificación

Herramienta / Tecnología	Propósito en el trabajo	Justificación de la elección
auditd	Generador de los logs que serán analizados por los modelos.	Es el estándar de auditoría sistemas Linux a nivel de kernel. Permite capturar eventos críticos de seguridad de manera estructurada, sirviendo como la fuente de datos primaria del proyecto.
ausearch	Herramientas nativas de filtrado que actúan como el baseline analítico del proyecto.	Permite contrastar los resultados del LLM contra los mecanismos tradicionales del sistema operativo.
Ollama	Motor de inferencia local encargado de gestionar la carga en memoria y la ejecución de los LLMs.	Simplicidad de uso y gestión automática del hardware. Al exponer una API REST local en el puerto 11434, facilita la integración con scripts.
phi3:mini	Modelo de lenguaje principal utilizado para la clasificación y asignación de niveles de riesgo.	La GPU del entorno de pruebas cuenta con una restricción de 4GB de VRAM, lo que impide ejecutar arquitecturas más grandes.

Herramienta / Tecnología	Propósito en el trabajo	Justificación de la elección
llama3.2:3b	Modelo de lenguaje secundario utilizado para el análisis comparativo de rendimiento.	Evaluar si las variaciones en el entrenamiento base y la arquitectura de un modelo del mismo rango de parámetros (3B) alteran la comprensión de logs. Permite contrastar si los aciertos o alucinaciones observados son particulares de phi3:mini o generalizables a los modelos más pequeños.
Python 3.12	Lenguaje de programación utilizado para el desarrollo del pipeline y de la optimización de los formatos.	Es el estándar de la industria para el desarrollo de pipelines de datos e IA. Permite estructurar la lógica de lectura de archivos, la transformación de RAW a JSON y la interacción directa con la API REST de Ollama mediante librerías nativas.
Pandas	Librería de Python empleada para el procesamiento estructurado y la analítica de métricas.	Facilita la manipulación de datos, permitiendo guardar los registros de salida en archivos CSV y automatizar el cálculo exacto de porcentajes de precisión y tiempos de respuesta por cada escenario evaluado.
Jupyter Notebook	Entorno de desarrollo interactivo para la documentación y análisis del proyecto.	Permite segmentar el pipeline de pruebas, visualizar los resultados de distintas formas y documentar los análisis paso a paso.
SciPy	Librería de Python empleada para el análisis de reproducibilidad y significancia estadística de las pruebas	Es el estándar para computación científica y estadística en el ecosistema de Python. Permite ejecutar pruebas complejas (como pruebas t de Student) y calcular intervalos de confianza.

6. Implementación y desarrollo

6.1 Diseño de la solución

El diseño de la solución se plantea como un pipeline automatizado de procesamiento secuencial de datos distribuidos entre dos entornos operativos (máquina virtual con Ubuntu Server 22.04 LTS y sistema anfitrión Windows). El objetivo principal es capturar eventos de bajo nivel del kernel, separarlos de la infraestructura auditada y procesarlos localmente mediante LLMs.

La solución se estructura en tres etapas secuenciales donde la primera etapa se lleva a cabo en el entorno Linux y las siguientes en Windows. En la primera etapa se realizó la captura de eventos y la generación de los datasets, llevada a cabo en una máquina virtual con Ubuntu Server 22.04 LTS. Se ejecutaron comandos que simulan escenarios distintos, el subsistema nativo *auditd* captura las llamadas al sistema en base a reglas preconfiguradas. Tras la ejecución de los escenarios de riesgo y ruido, con la herramienta *ausearch* se extraen los registros estructurados crudos (RAW) y se exportan al sistema anfitrión.

En la siguiente etapa, optimización de los datasets, mediante un script en Python el log en formato RAW es sometido a una reducción de ruido ya que incluye metadatos repetitivos, descartando los campos innecesarios y dejando solo los campos imprescindibles. Esto genera dos variantes de formato, RAW reducido y JSON reducido. Esta optimización de formatos se lleva a cabo no solo para comparar qué formato funciona mejor, si para reducir el consumo de tokens de los modelos y reducir la sobrecarga computacional de la GPU.

Por último se lleva a cabo la etapa de inferencia del LLM, diseñada bajo una granularidad unitaria (evento por evento) ya que con un hardware limitado (GPU con 4GB VRAM), la inyección de múltiples líneas de logs concurrentes en un único prompt saturaría de forma inmediata la ventana de contexto de los modelos más reducidos, aumentando las alucinaciones semánticas de las respuestas. Por lo tanto, el procesamiento unitario garantiza la estabilidad y viabilidad técnica de la infraestructura disponible. Para cada dupla escenario-formato, el pipeline lee secuencialmente cada registro del dataset y construye dinámicamente un prompt estructurado basado en la estrategia *few-shot prompting*. Se decidió utilizar esta estrategia ya que la inclusión explícita de pares de ejemplos estructurados de los logs y salidas correctas pueden ayudar al modelo a mantener una estructura sintáctica de la respuesta y elevar su precisión de clasificación. Este payload es enviado a través de una petición a la API local al motor de inferencia *Ollama* (puerto 11434). El modelo seleccionado es cargado en la GPU y procesa el token de entrada y devuelve una respuesta estructurada que contiene el nivel de riesgo (ALTO/MEDIO/BAJO/INFO) y su justificación técnica. El pipeline toma esta respuesta y la junta con el tiempo de respuesta, el log analizado y el modelo utilizado en un archivo CSV para su posterior análisis.

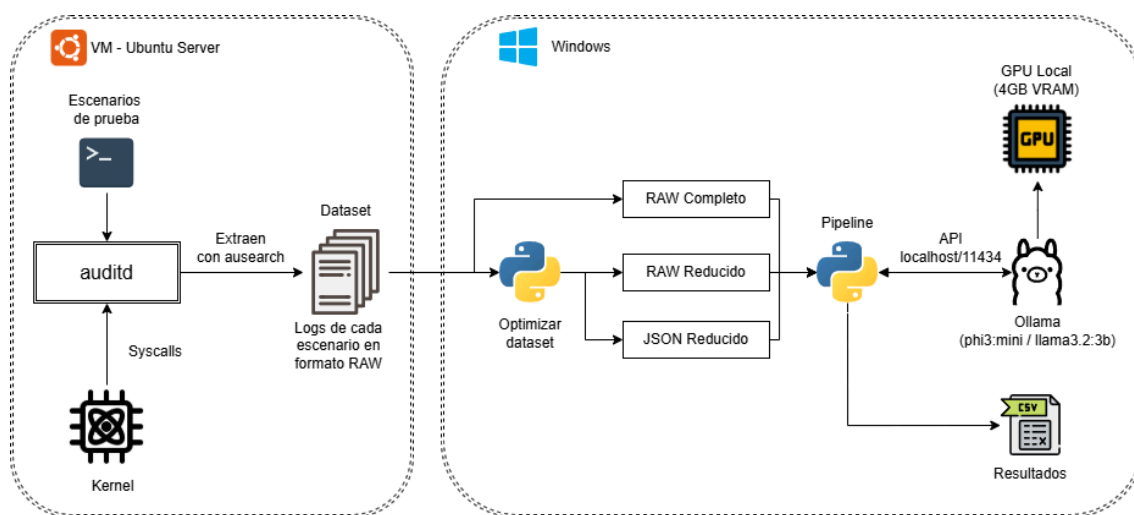


Figura 1. Arquitectura general del pipeline del procesamiento local de logs de auditoría mediante LLMs. Fuente: Elaboración propia.

6.2 Implementación

6.2.1 Generación y captura de eventos

Este componente opera en el espacio de kernel y de servicios del entorno virtualizado (*Ubuntu Server 22.04 LTS*). Su función es disparar de forma controlada comportamientos anómalos y rutinarios para compilar el dataset mediante el subsistema *auditd*. Se simularon cinco escenarios de evaluación y se extrajeron los logs con *ausearch*. Adicionalmente se generó un log de ruido.

Escenario 1 - Acceso normal a /etc/passwd

Se simuló la lectura normal de cuentas de usuario utilizando los siguiente comandos:

- *cat /etc/passwd*: Lectura completa del archivo.
- *tail -n 5 /etc/passwd*: Lectura de las últimas 5 líneas.
- *grep "root" /etc/passwd*: Búsqueda de un string específico dentro del archivo.

Los logs se extrajeron utilizando el siguiente comando:

```
sudo ausearch -k passwd_changes --raw > dataset/escenario1_raw.log
```

Se registraron 139 eventos.

Escenario 2 - Intento de escaladas de privilegios

Registra auditorías de fallos de autorización por parte de un usuario sin privilegios que intenta ejecutar comandos restringidos, generando eventos de denegación. Los comandos utilizados fueron:

- *sudo -l*: Intento de listar qué privilegios tiene el usuario actual, solicita contraseña y luego falla.
- *su - root*: Intento directo de cambiar a la sesión de root.

- *sudo cat /etc/shadow*: Intento de leer un archivo restringido usando sudo el cual falla por falta de permisos.

La extracción de los logs se ejecutó mediante:

```
sudo ausearch -m USER_CMD,USER_AUTH -sv no --raw > dataset/escenario2_raw.log
```

Se utilizó *-sv no* para capturar los intentos de escalada que el sistema rechaza, no solo los exitosos. Se registraron 7 eventos.

Escenario 3 - Ejecución de comando como root

Contiene registros de administración habituales del sistema. Se utilizaron los siguientes comandos:

- *sudo apt-get update*: Tarea de mantenimiento habitual y ruidosa.
- *sudo systemctl status auditd*: Consulta del estado de un servicio.
- *sudo useradd usuario_fantasma*: Creación de un usuario nuevo.

La extracción de los logs se ejecutó con el siguiente comando:

```
sudo ausearch -k root_commands --raw > dataset/escenario3_raw.log
```

Debido a la alta tasa de ruido provocado por el gestor de paquetes *apt-get*, este dataset crudo de 652 eventos iniciales fue acotado a los comandos mencionados anteriormente, para ello se filtraron los logs por *audit=1000* y por los *comm* que se utilizaron para generar el dataset: *apt-get*, *systemctl* y *useradd*. El resultado fue pasar de 652 eventos a solo 10.

Escenario 4 - Modificación de /etc/sudoers

Se registran alteraciones directas o de metadatos sobre el archivo de privilegios del sistema operativo. A diferencia de otros escenarios, se ejecutaron deliberadamente tres formas distintas de interacción (*visudo*, *touch* y *chmod*):

- *sudo visudo*: Edición real del contenido. Se utilizó para realizar una modificación física directa del archivo, añadiendo un comentario, *# Test TP*, al final del archivo. *Auditd* registra eventos que incluyen registros tipo *SYSCALL* (asociados a llamadas como *rename* u *openat*) y registros *PATH* con etiquetas *nametype=CREATE* y *nametype=DELETE*, evidenciando la mutación del contenido.
- *sudo touch /etc/sudoers*: No modifica el contenido, pero actualiza la marca de tiempo de modificación. Desde la óptica de *auditd*, el evento no se clasifica como una alteración de datos corporales sino como una modificación de atributos.
- *sudo chmod 0440 /etc/sudoers*: Se aplicó para reafirmar los permisos de solo lectura del archivo lo cual modifica los metadatos del archivo. Este comando invoca la *syscall fchmodat*. Al igual que el comando anterior, *auditd* lo procesa como un cambio de atributos de seguridad. Sin ser una escritura de datos el subsistema registra la mutación de los bits de modo del archivo, reflejando el cambio directamente en el campo *mode* del registro *PATH* (*mode=010040*).

Los logs se extrajeron con el siguiente comando:

```
sudo ausearch -k sudoers_changes --raw > dataset/escenario4_raw.log
```

Se registraron 76 eventos.

Escenario 5 - Acceso a /etc/shadow por proceso no autorizado

Se registraron intentos de accesos a las contraseñas del sistema, los comandos fueron ejecutados por un usuario sin privilegios. Los comandos utilizados fueron:

- `cat /etc/shadow`: Un intento de lectura directo
- `head -n 1 /etc/shadow`: Intento de leer solo la primera línea, donde suele estar la clave de root.
- `cp /etc/shadow /tmp/shadow_copy`: Intento de copiar el archivo a un directorio público.

La extracción de logs se realizó ejecutando el comando

```
sudo ausearch -k shadow_access --raw > dataset/escenario5_raw.log
```

Se registraron 243 eventos.

Log de ruido

Para conseguir medir los falsos positivos, se generó un log de ruido. Para ello primero se reinició la máquina virtual, se esperaron 10 minutos sin ejecutar ningún comando, esto para asegurarse de que no entren los comandos de los otros escenarios y se procedió a realizar tareas comunes y rutinarias como entrar a directorios, crear archivos, editar archivos no críticos. Se registraron 475 eventos.

Se extrajeron los logs con el siguiente comando:

```
sudo ausearch --start recent --raw > dataset/ruido_raw.log
```

6.2.2 Baseline

Con el fin de tener un *baseline* con el cual comparar los resultados de los modelos se definieron reglas manuales de búsqueda por palabras clave y se las aplicó sobre los archivos de los logs generados para cada escenario. Los comandos utilizados fueron:

- `grep -i "passwd" dataset/escenario1_raw.log > baseline/escenario1_baseline.log`
- `grep -E -i "res=failed|success=no" dataset/escenario2_raw.log > baseline/escenario2_baseline.log`
- `grep -E -i 'comm="useradd"|comm="apt-get"|comm="systemctl"' dataset/escenario3_raw.log > baseline/escenario3_baseline.log`
- `grep -i "sudoers" dataset/escenario4_raw.log > baseline/escenario4_baseline.log`
- `grep -i "shadow" dataset/escenario5_raw.log > baseline/escenario5_baseline.log`

Todos los *baseline* detectan correctamente las ejecuciones de los escenarios, pero en todos los casos, menos en los escenarios 2 y 3 que tienen pocos eventos, también capturan muchos logs de ruido. Por ejemplo, para el escenario 4 capturó eventos *CONFIG_CHANGE* y lecturas normales, y en para el escenario 5 capturó muchas autenticaciones normales (*sshd*, *login*, *systemd*, *cron*).

6.2.3 Optimización de los datasets

Se desarrolló un script con Python para reducir el consumo de tokens de los modelos a la hora de leer los eventos. El script evalúa las líneas del log en formato RAW y extrae únicamente 11 campos esenciales. Los campos retenidos son de dos tipos: descriptores de identidad (*uid*, *euid*, *gid*, *auid*), e indicadores operacionales de la llamada al sistema (*syscall*, *success*, *exit*, *comm*, *exe*, *key* y *name* para los eventos *PATH*), preservando en la cabecera los campos estructurales *type* y *msg=audi*. Los eventos que no tienen ninguno de estos campos mencionados, se mantienen en los nuevos formatos para no perder información pero serán omitidos en el análisis de los modelos.

Para finalizar, se exporta la información procesada en dos variantes de formato para luego analizar, el formato RAW Reducido mantiene la sintaxis lineal nativa de clave-valor separada por espacios, pero habiendo eliminado todos los campos descartados por el filtro, mientras que el JSON Reducido transforma la estructura lineal depurada en un objeto JSON clave-valor estructurado para el análisis semántico de los modelos de lenguaje.

6.2.4 Modelos y motor de inferencia

El motor de ejecución utilizado es Ollama, un servicio local que expone una interfaz de programación de aplicaciones (*API REST*) en el puerto local 11434, permitiendo desacoplar el pipeline de dependencias externas en la nube. Los modelos que componen el estudio comparativo son, *phi3:mini* desarrollado por Microsoft de 3.800 millones de parámetros y *llama3.2:3b* desarrollado por Meta que cuenta con 3.000 millones de parámetros.

La interacción con Ollama se realiza mediante sockets locales HTTP enviando peticiones síncronas bajo el método *POST* al endpoint */api/generate*. En el cuerpo del mensaje (*payload*) se le envía el modelo y prompt a utilizar. Además se fuerza al modelo a operar de forma determinista desactivando el comportamiento de respuestas progresivas ("*stream*": *false*), lo cual estabiliza la recolección de las métricas temporales de rendimiento del hardware y se fuerza a que la respuesta la devuelva en formato JSON para facilitar la respuesta ("*format*": "*json*") para facilitar el parseo.

6.2.5 Inferencia y prompts

El pipeline en Python lee secuencialmente el dataset seleccionado (en el formato seleccionado) registro por registro. En los formatos reducidos existen eventos que quedaron sin ningún campo a analizar debido al proceso de optimización, en esos casos el registro es omitido del análisis y es agregado de manera automática al CSV con la clasificación de INFO y aclarando en la justificación que fue omitido del análisis del modelo. Por cada evento de auditoría procesado, se acopla el log crudo dentro de una estructura de instrucción basada en la técnica de *few-shot prompting*. La estructura lógica interna aplicada para empaquetar el prompt de entrada consta de tres secciones.

En la primera sección se define el rol del modelo de lenguaje como un experto en ciberseguridad y auditoría de sistemas Linux, acotando su tarea y el espacio de salida estrictamente a los cuatro niveles de riesgo (ALTO, MEDIO, BAJO, INFO) y una justificación en una línea, aclarándole que su respuesta tiene que ser en formato JSON. Además se le agrega una regla de no usar comillas dobles (") dentro del texto de justificación, esto para evitar problemas con el JSON y futuros análisis.

En la segunda sección tenemos los pares de ejemplos estructurados que se inyectan casos preestablecidos de logs de entrada reales junto con sus clasificaciones de riesgo y justificaciones.

En la última sección, se introduce al final de la secuencia el log individual que el pipeline está evaluando en ese instante, cerrando la instrucción para forzar la predicción del modelo.

Tras recibir el payload estructurado de la API de Ollama, el orquestador decodifica la cadena JSON devuelta, parsea las variables semánticas e indexa de forma limpia la información en un archivo CSV. Los campos de este archivo son: el log original, el nivel de riesgo clasificado por el modelo, la justificación del modelo, el tiempo de respuesta en segundos y el modelo que realizó la evaluación.

6.3 Análisis según modelo OSI o capas del SO

Capa de Hardware

Esta capa actúa como la restricción física del proyecto. Si bien no genera las cadenas de texto del log, el hardware determina de manera directa las variables dependientes del experimento: el tiempo de respuesta analítico por log (segundos por inferencia) y la tasa de transferencia de tokens, debido a los cuellos de botella generados por la carga de los parámetros de los modelos en los 4GB de VRAM. Los mecanismos del sistema operativo involucran la gestión y asignación de memoria física mediante la MMU (Memory Management Unit), el manejo de interrupciones de hardware y las interfaces físicas que luego serán abstraídas por los controladores de la GPU en la capa superior.

Capa de Kernel

Esta capa es el motor generador de los dataset. Cada escenario de ataque o ruido simulado en el espacio de usuario fuerza al sistema a ejecutar *syscalls* específicas. El kernel captura y estructura los campos que componen las líneas de los registros. En cuanto a los mecanismos del sistema operativo involucrados, se destacan las llamadas al sistema, mediante las cuales los procesos en el espacio de usuario interactúan con los recursos de hardware, solicitando operaciones al kernel a través de interrupciones de software (tales como *open*, *execve*, *chmod*, *setuid*, *read* y *write*). Asimismo, es en esta capa donde operan y residen los controladores (drivers) de la GPU. Estos se encargan de servir como traductores entre el software y la placa de video, gestionando el acceso a la VRAM y traduciendo las peticiones del espacio de usuario en instrucciones físicas de procesamiento.

Capa de Servicios de sistema

Esta capa se encarga de filtrar y exportar grandes volúmenes de logs hacia archivos de texto plano legibles por aplicaciones externas. En este proceso, el demonio *auditd* se ejecuta en el espacio de usuario con privilegios de superusuario para interceptar los eventos de auditoría del núcleo y almacenarlos de manera persistente en */var/log/audit/audit.log*. Finalmente, *ausearch* complementa este flujo, permitiendo abstraer consultas complejas a través de llaves de auditoría.

Capa de Aplicaciones

En esta capa reside la lógica analítica de la solución. El pipeline desarrollado en Python procesa secuencialmente el archivo registro por registro, construye el prompt

inyectando el log a analizar, automatiza las llamadas síncronas a la API de Ollama y recolecta las respuestas del LLM en un archivo CSV para su posterior análisis. En relación a los mecanismos del sistema operativo, intervienen procesos convencionales ejecutados en el modo usuario (Ring 3), así como la gestión de memoria virtual asignada tanto al runtime de Python como a los hilos de Ollama. Finalmente, la comunicación entre el pipeline y el LLM se realiza de forma interna mediante peticiones HTTP sobre la interfaz local.

6.4 Pruebas y verificación

Para validar el pipeline y el comportamiento de los modelos, se diseñaron una serie de pruebas. Esta sección describe el marco de evaluación, la lógica de comprobación basada en una verdad de referencia (*Ground Truth*) y qué fue lo que se probó. Los resultados de las pruebas se pueden ver en el apartado 7. *Resultados y análisis*.

6.4.1 Marco de evaluación y Ground Truth

La verificación del pipeline no se realizó mediante inspección visual manual, sino desarrollando un módulo complementario en *Jupyter Notebook*, utilizando la librería *pandas*. Este módulo actúa sobre los archivos CSV contrastando las clasificaciones de riesgo emitidas por el LLM frente a las reglas del Ground Truth definidas para la investigación. Las reglas definidas son las que se presentan a continuación.

Para el escenario 1 (acceso a */etc/passwd*) se definieron las siguientes reglas: si el tipo de evento es una llamada al sistema (*TYPE=SYSCALL*) Y el comando (*comm*) coincide con utilidades de lectura (*cat*, *tail*, *grep*) ejecutadas por el usuario con *uid=1000*, la calificación esperada es BAJO o INFO debido a que corresponde a un acceso normal. Si en el evento *SYSCALL* el comando involucra la elevación intermedia de credenciales (*comm=sudo*) con una identidad efectiva de superusuario (*euid=root*) se espera un riesgo MEDIO. Los eventos *CWD* y *PROCTITLE* se espera una clasificación de INFO ya que no contienen información de riesgo. Los eventos restantes se esperan que sean clasificados como BAJO o INFO ya que representan ruido.

Se definieron las siguientes reglas para el escenario 2 (intento de escalada de privilegios). Si el registro almacena el comando hexadecimal correspondiente al intento de lectura del archivo confidencial de contraseñas (*CMD=636174202F6574632F736861646F77*, mapeado como *cat /etc/shadow*) se espera una clasificación de riesgo ALTO. Para registros de autenticación fallida (*USER_CMD* o *USER_AUTH* con estado *RES=FAILED*) la clasificación esperada es MEDIO o ALTO. Los eventos restantes se esperan que sean clasificados como BAJO o INFO ya que representan ruido.

Con respecto al escenario 3 (ejecución de comandos como root), eventos del tipo *SYSCALL* que involucren actividades rutinarias ejecutadas por la identidad del superusuario (*UID=ROOT* junto con *comm=apt-get* o *comm=systemctl*) se validan bajo etiquetas INFO o BAJO debido a que representan tareas de mantenimiento de la infraestructura. Mientras que la ejecución del comando de persistencia y alteración de cuentas locales (*comm=useradd*) realizado por un usuario con privilegios se espera una clasificación de MEDIO.

En el escenario 4 (modificación de */etc/sudoers*), si el evento es una llamada al sistema y es ejecutada por componentes vinculados directamente con la alteración o

el cambio de atributos del archivo de privilegios (*comm=visudo*, *comm=touch*, *comm=chmod*) se espera una clasificación de ALTO o MEDIO y en si es ejecutada por *comm=sudo* se espera que sea clasificado como MEDIO ya que puede requerir auditoría pero no es crítico. Para eventos del tipo *PATH* que indiquen la eliminación o creación de archivos relacionados con *sudoers* (*nametype=delete*, *nametype=create*) la clasificación esperada es MEDIO o ALTO. Los eventos restantes se esperan que sean clasificados como BAJO o INFO ya que representan ruido.

Para finalizar, las reglas del escenario 5 (acceso a */etc/shadow*) se definieron de la siguiente manera: si se interceptan llamadas al sistema denegadas por el kernel debido a la falta de permisos en el espacio de usuario (*comm=cat*, *head*, *cp* operando sobre */etc/shadow* y devolviendo un código de salida de error por falta de privilegios *EXIT=-13*) se espera que sea clasificado como ALTO. Si usuarios no autorizados (*UID > 1000*) invocan comandos de alteración de identidad (*comm=sudo*, *comm=su*), el riesgo esperado es ALTO o MEDIO. Para eventos tipo *SYSCALL* con *comm=sshd*, *login*, (*systemd*) y *cron* se deben clasificar como INFO O BAJO ya que corresponden a la autenticación de usuarios por parte del sistema operativo. Eventos *SYSCALL* con *audi=1000* utilizando el comando *sudo* (*comm=sudo*) se espera una clasificación de BAJO o INFO, ya que corresponden a la extracción de los logs con *ausearch* realizada para el trabajo. Por último los eventos restantes se esperan que sean clasificados como BAJO o INFO ya que representan ruido.

Tras recorrer cada fila del CSV de salida, el módulo calcula automáticamente el porcentaje de precisión global por escenario mediante la siguiente ecuación matemática:

$$\text{Precisión} = \left(\frac{\text{Eventos clasificados correctamente}}{\text{Eventos totales evaluados}} \right) \times 100$$

6.4.2 LLM vs Baseline

Con el propósito de evaluar el desempeño del LLM frente al baseline establecido, constituido por reglas manuales mediante la herramienta *ausearch*, se cuantificó inicialmente la precisión del modelo *phi3:mini* utilizando el formato RAW completo en combinación con el Prompt 1 (cuyos detalles se encuentran en la sección 6.4.4 *Pruebas de prompts*) sobre todos los escenarios. Sobre esta base, se procedió con un análisis de falsos positivos y falsos negativos identificados en los distintos escenarios. Finalmente, se efectuó un contraste directo entre las capacidades de detección de eventos del LLM y las del *baseline*, permitiendo determinar la efectividad relativa de ambos enfoques de auditoría.

6.4.3 Pruebas de formatos

Con el fin de analizar qué formato tiene mejores resultados, utilizando el modelo *phi3:mini* y el Prompt 1, se evaluaron los tres formatos de logs (RAW completo, RAW reducido y JSON reducido) a lo largo de los cinco escenarios. El análisis comparó la precisión, el tiempo promedio de respuesta por escenario y la distribución de las clasificaciones de riesgo obtenidas.

Debido a la reducción de campos aplicada en los formatos reducidos, y con el objetivo de garantizar una validación rigurosa frente al Ground Truth, se utilizó el dataset en formato RAW completo (perteneciente al mismo escenario de análisis) para armar una guía con las clasificaciones esperadas para cada evento. A partir de esta referencia, se contrastaron las predicciones obtenidas en los formatos reducidos. Asimismo, dado

que los logs omitidos en el proceso de optimización fueron clasificados automáticamente como INFO, estos eventos se excluyeron del cálculo de la precisión final. De este modo, se evita distorsionar la métrica y se obtiene un reflejo exacto del rendimiento del modelo sobre los escenarios.

6.4.4 Pruebas de prompts

Con el propósito de evaluar qué estructura de directivas proporciona un rendimiento óptimo en el análisis de eventos de *auditd*, se diseñaron y probaron tres variantes de *prompts*, todas bajo la estrategia de *few-shot prompting*. Los textos íntegros de los prompts se encuentran disponibles para su consulta en el *Anexo A – Prompts*.

Las tres variantes comparten un esqueleto estructural idéntico, difiriendo únicamente en los ejemplos de referencia provistos al modelo. La arquitectura base inicia con una asignación de rol, definiendo al LLM como un experto en ciberseguridad y análisis de logs en sistemas Linux. Posteriormente, se delimita la tarea de análisis del registro y se imponen restricciones estrictas sobre la salida, exigiendo un formato JSON que contenga la clasificación del riesgo y su correspondiente justificación en una sola línea, respetando además la regla de omitir comillas dobles internas detallada previamente. Finalmente, se presentan los casos de ejemplo seguidos del log objetivo a procesar.

El *Prompt 1* se configuró con dos ejemplos (clasificaciones BAJO y ALTO) y operó como la directiva principal, ejecutándose de forma transversal sobre la totalidad de los escenarios y formatos del proyecto. El *Prompt 2* incorporó un total de cuatro ejemplos, uno para cada nivel de riesgo, con el fin de determinar si un mayor volumen de contexto optimiza la precisión del modelo. Por último, el *Prompt 3* se estructuró con tres ejemplos (INFO, MEDIO y ALTO) a los que se les añadieron reglas explícitas con criterios de clasificación para los eventos. Estas dos últimas variaciones experimentales se evaluaron específicamente sobre los escenarios 1, 2 y 5 dentro de los formatos RAW completo y RAW reducido.

6.4.5 Pruebas de modelos

Con el propósito de evaluar el impacto de la arquitectura del modelo tanto en la precisión de las clasificaciones como en el tiempo de respuesta, se llevó a cabo un análisis comparativo entre dos LLMs locales. El modelo base del proyecto fue *phi3:mini*, sobre el cual se ejecutó la totalidad de la matriz de pruebas, abarcando todos los escenarios y formatos definidos. Por otro lado, el modelo *llama3.2:3b* con el cual se ejecutaron todos los escenarios en el formato RAW Completo.

6.4.6 Pruebas de reproducibilidad y significancia estadística

Con el objetivo de demostrar que los resultados obtenidos no son producto de una anomalía estadística o un accidente de una única corrida, se diseñó una prueba de reproducibilidad. Se ejecutó el pipeline en 5 iteraciones independientes sobre los mismos escenarios, utilizando el modelo *phi3:mini* en formato RAW completo con el *Prompt 1*. Posteriormente se calculó el promedio de precisión, el desvío estándar y el intervalo de confianza del 95% para cada escenario, demostrando la estabilidad del modelo.

Finalmente, para justificar la elección del modelo, se sometió al modelo secundario *llama3.2:3b* al mismo rigor de 5 iteraciones y se aplicó un análisis de significancia estadística mediante la prueba t de Student. Se estableció un valor $p < 0.05$ para determinar si la diferencia de precisión entre ambos modelos frente a los mismos logs

representa una superioridad real de una de las arquitecturas, o si las variaciones podrían ser simplemente producto del azar.

6.4.7 Pruebas de logs de ruido

Con el propósito de determinar con qué precisión el LLM es capaz de clasificar registros de actividad normal y cuantificar el volumen de falsos positivos generados, se realizó una evaluación específica sobre el dataset de logs de ruido utilizando el modelo *phi3:mini*. Para esta prueba se seleccionó el formato RAW reducido (compuesto por 244 eventos) sobre el formato RAW completo (de 475 eventos), debido a la elevada carga computacional y la densidad de registros que este último presentaba. Este análisis resulta fundamental para el desarrollo del trabajo, ya que evaluar la tasa de falsos positivos ante el comportamiento cotidiano y de fondo del sistema operativo permite determinar la viabilidad operativa real del LLM, garantizando que la herramienta no sature a los administradores con alertas innecesarias.

7. Resultados y análisis

LLM vs Baseline

Las reglas manuales de *ausearch* demuestran una detección del 100% pero carecen por completo de especificidad analítica. Al filtrar mediante palabras claves también recolecta demasiados logs de ruido lo que obliga a un administrador a interpretar de forma manual el riesgo de los logs.

Como se puede observar en la Tabla 1, el LLM logra clasificar con éxito el riesgo de los escenarios 1, 2, 4 y 5, manteniendo una tasa de precisión estable por encima del 73% bajo el formato *RAW Completo*. En el escenario 1 no registró ningún falso negativo, esto quiere decir que todos los errores fueron falsos positivos, en general exagero los comandos *sudo* los cuales debía clasificar como MEDIO y los clasificó como ALTO, las tareas de mantenimiento (comandos de lectura ejecutados por *uid=100*) que debían ser INFO/BAJO los clasificó como MEDIO.

Sobre el escenario 2 solo clasificó mal un evento de los 7, pero este era un evento crítico, ya que corresponde al *cmd=636174202F6574632F736861646F77* el cual se traduce en *cat /etc/shadow*. El modelo no decodificó el hexa y clasificó como MEDIO cuando tendría que ser ALTO.

En el escenario 3 es donde el LLM tuvo su peor rendimiento cayendo al 10% de precisión, esto se debe a que exageró comandos legítimos de administración ejecutados por un usuario *root* (como *sudo apt-get update*), clasificando erróneamente todos los escenarios menos unos, asignándoles un nivel de riesgo ALTO cuando debían ser MEDIO.

Si bien el LLM tuvo un 73.68% de precisión sobre el escenario 4, generó un falso negativo importante: clasificó como INFO la ejecución de *visudo* (cuando debería ser MEDIO/ALTO) y su justificación fue “El comando visudo se está ejecutando con permisos completamente administrativos, lo cual es esperado para modificar archivos de *sudoers*.”, es decir que el modelo “razonó” demasiado y asumió que era un mantenimiento legítimo, perdiendo el contexto de la amenaza. El resto de los errores corresponden a falsos positivos que el modelo tendió a sobre-alertar en eventos de bajo riesgo. Por ejemplo clasificó un evento *PROCTITLE* (el cual correspondía a la ejecución de *ausearch*) como MEDIO, con una justificación “Examinando el log, se observa una operación de copia completa del directorio */etc/shadow* hacia un archivo temporal con permisos incorrectos que podría indicar intento de extraer información sensible.”, en la cual inventó un comportamiento malicioso a partir de una cadena de texto inofensiva.

Con respecto al escenario 5 el modelo detectó correctamente todos los eventos críticos (*cat*, *head*, *cp sobre /etc/shadow*) pero generó un total de 62 falsos positivos, al igual que en otros escenarios, tiende a sobreestimar acciones legítimas administrativas.

Como se puede observar en la Tabla 1, el tiempo promedio de respuesta de cada escenario oscila entre los 5.86 segundos y los 7.96 segundos promediando un 6.69 segundo de análisis por log. Esta diferencia entre el escenario 5 y 2 se puede deber a que el primer log que analiza el modelo siempre tarda más ya que primero se carga el modelo en la GPU y luego realiza la tarea, y como el escenario 5 tiene 234 evento está tiempo extra inicial es menos significativo que en los escenarios con menos eventos.

En conclusión, existe coincidencia entre la detección de los eventos en todos los escenarios exceptuando el 3 y los dos casos mencionados anteriormente, con la contraparte de que las reglas manuales generan logs de ruido y el LLM genera falsos positivos.

Escenario	Eventos Totales	Baseline Detecta	LLM Detecta	Precisión	Tiempo Promedio (s)
1	139	SI	SI	73.38%	6.15
2	7	SI	SI	85.71%	7.06
3	10	SI	NO	10.00%	7.96
4	76	SI	SI	73.68%	6.41
5	234	SI	SI	73.08%	5.86

Tabla 1. Comparación LLM vs Baseline. Se muestra para cada escenario la precisión y tiempo promedio de respuesta medida en segundos del modelo phi3:mini en el formato RAW completo y la comparación con el baseline de detección.

Formatos

En la Tabla 2, podemos observar como la estructura sintáctica y la densidad informativa de la entrada alteran drásticamente el rendimiento analítico de un LLM compacto como lo es *phi3:mini*. El formato RAW completo mantiene la precisión más equilibrada y elevada en la mayoría de los escenarios. Esto denota que el modelo asimila mejor la sintaxis lineal nativa clave-valor de *auditd* separada por espacios, mapeando dependencias semánticas complejas si no se alteran los metadatos contextuales generados por el kernel.

El formato JSON reducido desploma las precisiones a rangos deficientes de entre el 34% y 35%, exceptuando el escenario 2. La introducción de caracteres de control estructurados (llaves, comillas dobles) satura la ventana de atención del modelo. El hardware gasta tokens esenciales decodificando la estructura del string JSON en lugar de razonar sobre las implicancias de ciberseguridad del log de auditoría.

Con el formato RAW reducido el modelo actúa de forma contradictoria según el escenario. En el Escenario 3, la remoción de metadatos repetitivos limpió el ruido semántico, permitiendo al modelo enfocarse en la llamada limpia y elevar su precisión del 10.00% al 70.00%. Sin embargo, en los escenarios de escalada e intrusión (Escenarios 2, 4 y 5), la eliminación de metadatos críticos impidió al modelo correlacionar el contexto, provocando una caída de la precisión, como por ejemplo, del 73.08% al 35.29% en el escenario 5.

Formato	Escenario 1	Escenario 2	Escenario 3	Escenario 4	Escenario 5
RAW Completo	73.38%	85.71%	10.00%	73.68%	73.08%
RAW Reducido	74.29%	28.57%	70.00%	53.66%	47.06%
JSON Reducido	34.29%	85.71%	10.00%	34.15%	35.29%

Tabla 2. Precisión de los formatos. Se muestra para cada formato la precisión del modelo ϕ_3 :mini en cada escenario.

La Figura 2 detalla las variaciones de la latencia media de inferencia (segundos por log) en función del formato de entrada procesado. Teóricamente, la reducción implementada en los formatos *RAW Reducido* y *JSON Reducido* debió acortar linealmente los tiempos de procesamiento al procesar menos tokens de entrada. Sin embargo, las gráficas evidencian que los tiempos se mantienen en una meseta rígida de entre 5.8 y 7.9 segundos entre escenarios.

Esto demuestra que en arquitecturas de cómputo limitado (GPU de 4GB de VRAM), el cuello de botella temporal no está determinado por la longitud del prompt de entrada, sino por el costo fijo de cargar y activar la matriz de pesos del modelo local en la memoria de video y por el sobreesfuerzo que sufre el modelo al intentar parsear estructuras compactas o abstractas.

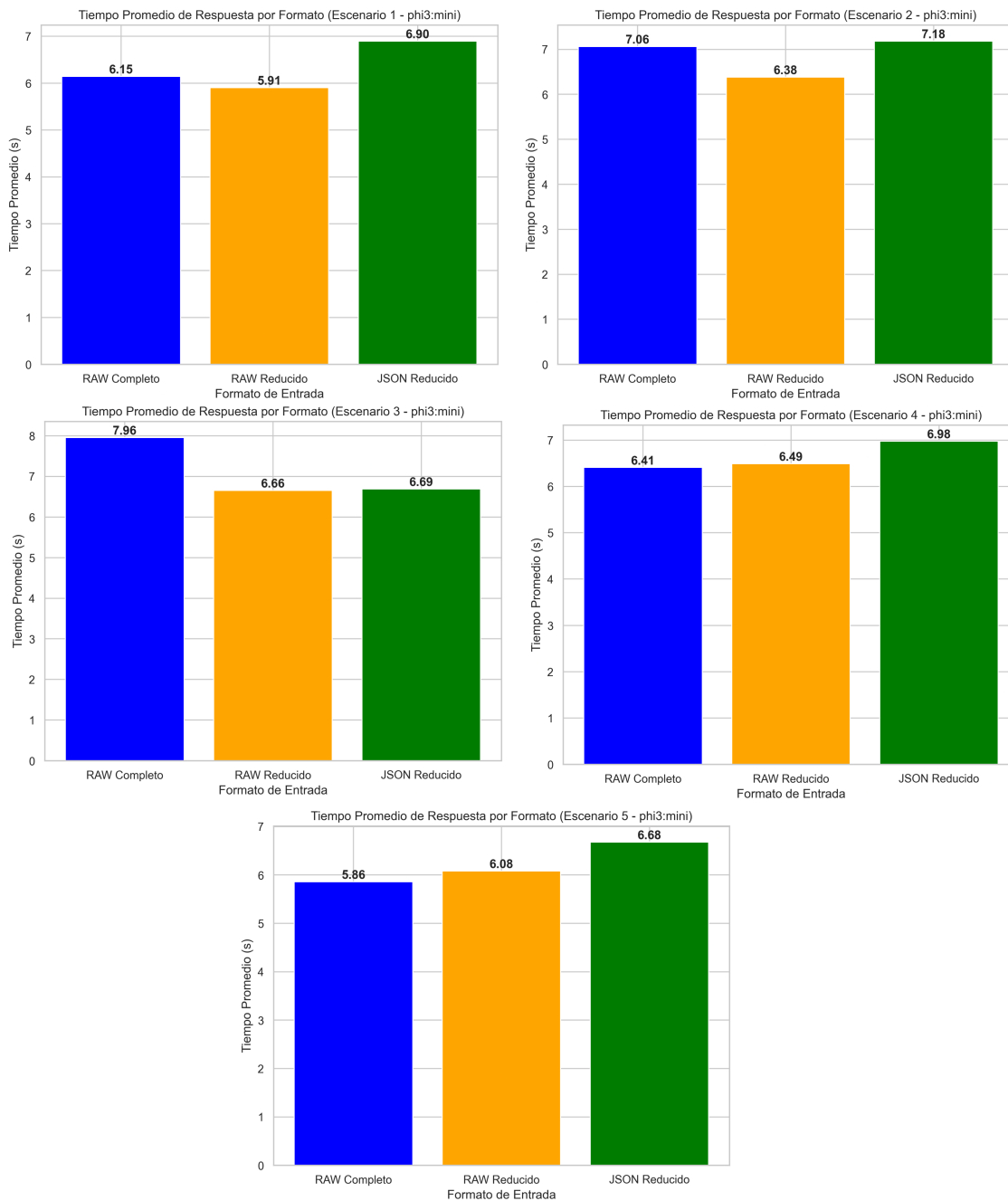


Figura 2. Tiempo promedio de respuesta por formato. Se muestra para cada escenario el tiempo promedio de respuesta medido en segundos para cada formato. Modelo utilizado phi3:mini.

La distribución de riesgos expuesta en la Figura 3 se puede observar que en los formatos reducidos (*RAW Reducido* y *JSON Reducido*), existe un incremento en la columna de clasificación INFO. Esto es una consecuencia directa del diseño del pipeline: todos los eventos omitidos durante la optimización del script en Python fueron catalogados automáticamente con dicha etiqueta. Para los escenarios 1, 4 y 5 se omitieron 69, 35 y 155 eventos respectivamente, mientras que en los escenarios 2 y 3 no se omitió ningún escenario.

Además, se puede observar como en el escenario 3 pasa de clasificar seis eventos como ALTO y uno MEDIO en RAW completo a no clasificar ninguno como ALTO y distribuirlo entre INFO/MEDIO en RAW reducido. Estas diferencias se pueden notar en todos los escenarios a mayo o menor medida.

También se puede notar como en los escenario 1 y 5 el modelo clasificó como INFORMACIÓN e INFORMADO eventos que debía clasificar como INFO, lo que demuestra que en algunas situaciones el modelo se sale de la instrucción clara de cómo clasificar los riesgos. Por último, en los escenarios 1 y 4 el modelo generó errores de parseo en el pipeline debido a saltarse la regla de no usar comillas dobles (") en la justificación.

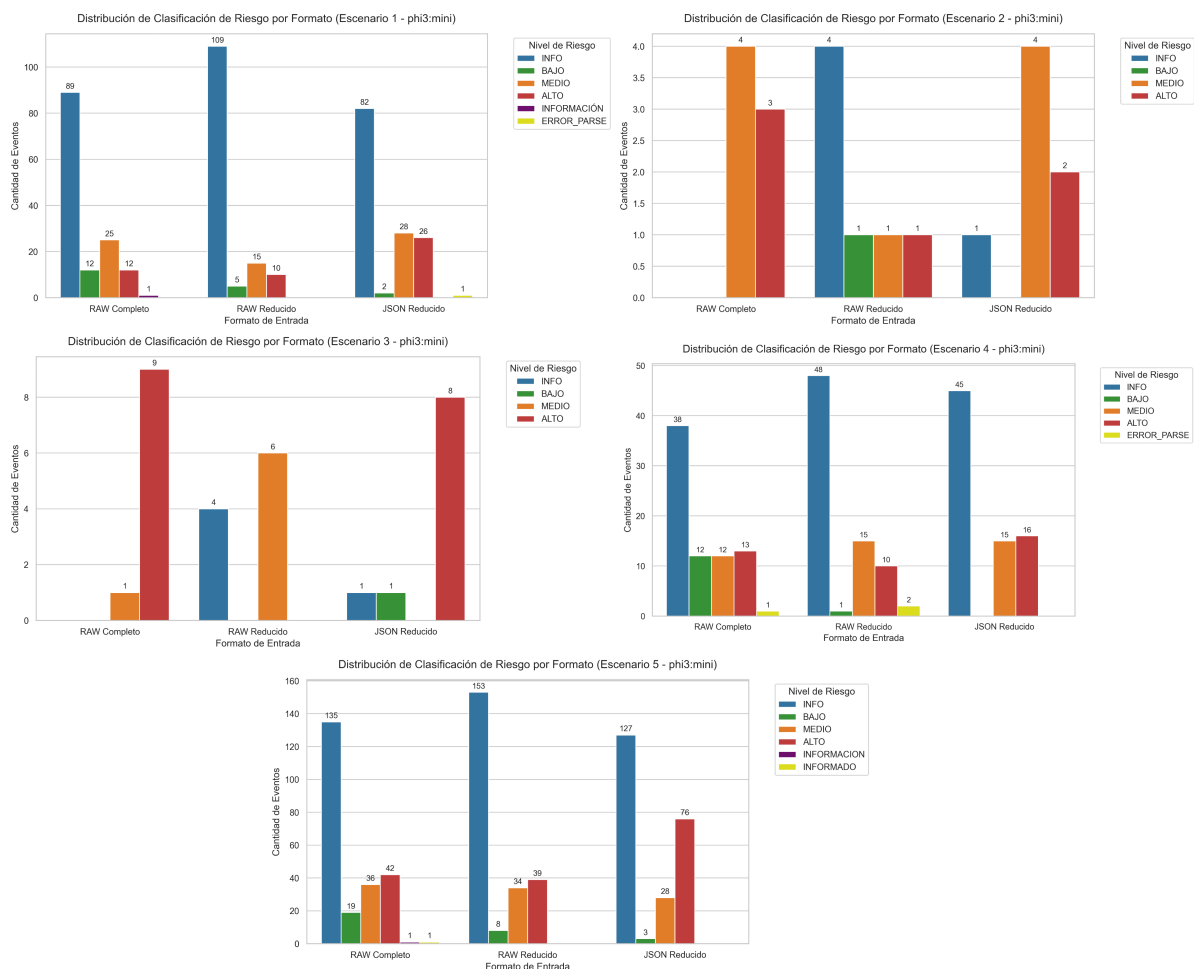


Figura 3. Distribución de riesgos. Se muestra para cada escenario la distribución de la clasificación de los riesgos para cada formato. Modelo utilizado: phi3:mini

Prompts

Como se muestra en la Tabla 3 al proveer únicamente dos ejemplos analíticos referenciales (BAJO y ALTO), el *Prompt 1* resguarda el espacio de contexto del modelo, logrando la mayor estabilidad interpretativa en volúmenes masivos de datos en el formato RAW completo.

Al expandir el prompt a cuatro ejemplos en el *Prompt 2* (uno para cada nivel de riesgo), el modelo logra un óptimo absoluto del 100.00% de precisión en el Escenario 2, tanto en formato completo como reducido. No obstante, al aplicar esta directiva extensa sobre escenarios con mayor población (escenarios 1 y 5), la precisión se desploma, cayendo al 42.45% en el escenario 1 (RAW completo) por ejemplo. Esto demuestra que en modelos compactos de menos de 4B de parámetros, los prompts largos saturan la ventana de atención activa, diluyendo las instrucciones e incrementando las alucinaciones semánticas.

El *Prompt 3*, que inyecta criterios explícitos de clasificación junto a tres ejemplos estructurados, demostró su máxima eficiencia al actuar sobre el formato RAW Reducido, alcanzando un 85.71% de precisión en el Escenario 1 y mejorando los resultados del *Prompt 2* en los escenarios 1 y 3 en ambos formatos. Sin embargo en general sus resultados están por debajo del *Prompt 1*.

Escenario	Prompt 1	Prompt 2	Prompt 3	Cantidad de eventos
1	73.38%	42.45%	46.04%	139
2	85.71%	100.00%	85.71%	7
5	73.08%	43.59%	71.37%	254

Tabla 3. Precisión de prompts en formato RAW completo. Se muestra la precisión de cada prompt para los escenarios 1, 2 y 5 y la cantidad de eventos de cada escenario. Modelo utilizado: phi3:mini.

Escenario	Prompt 1	Prompt 2	Prompt 3	Cantidad de eventos
1	74.29%	48.57%	85.71%	70
2	28.57%	100.00%	57.14%	7
5	47.06%	16.81%	35.29%	119

Tabla 4. Precisión de prompts en formato RAW reducido. Se muestra la precisión de cada prompt para los escenarios 1, 2 y 5 y la cantidad de eventos de cada escenario. Modelo utilizado: phi3:mini.

Versión final de prompt

Observando y analizando los resultados de las tres variantes de prompts, la mejor versión del prompt es la utilizada en el *Prompt 1* que incluye dos ejemplos, uno de clasificación BAJO y otro de clasificación ALTO. Esto se debe a que la instrucción es clara y a la vez no satura el contexto del modelo al ser más corto que las otras dos variantes. El texto completo del *Prompt 1* se encuentra disponible en el *Anexo A — Prompts*.

Estabilidad y reproducibilidad de resultados

Para confirmar que la capacidad de detección del modelo no es un accidente de una sola corrida, se evaluó la estabilidad de `phi3:mini` mediante la repetición del experimento (5 corridas independientes) sobre el formato de mayor rendimiento (RAW Completo) utilizando el *Prompt 1*.

Como se puede observar en la Tabla 5, los resultados demuestran de manera concluyente que el rendimiento del pipeline es estable y reproducible. El promedio de precisión de los escenarios 2, 4 y 5 se mantienen por encima del 70%, mientras que el del escenario 1 cae su precisión al 64%, demostrando la tendencia a generar falsos positivos. Por otro lado, el escenario 2 presentó la misma precisión en todas las repeticiones (en todas clasificó mal el mismo evento) y el escenario 3 muestra ser el que peor resultado obtuvo con todas sus repeticiones por debajo del 20% (algunas 0%). El desvío estándar se mantiene notablemente bajo, oscilando entre un 0.00% en el Escenario 2 y un máximo de 8.37% en el Escenario 3. Particularmente en los escenarios de mayor volumen de registros críticos (Escenarios 4 y 5), la desviación es mínima (2.53% y 1.51% respectivamente).

	Escenario 1	Escenario 2	Escenario 3	Escenario 4	Escenario 5
Repetición 1	73.38%	85.71%	10.0%	73.68%	73.08%
Repetición 2	60.43%	85.71%	0.0%	75.0%	72.65%
Repetición 3	63.31%	85.71%	20.0%	71.05%	70.09%
Repetición 4	61.87%	85.71%	10.0%	72.37%	69.66%
Repetición 5	62.59%	85.71%	0.0%	68.42%	71.37%
Promedio	64.32%	85.71%	8.0%	72.1%	71.37%
Desvío Est.	5.18%	0.0%	8.37%	2.53%	1.51%
Interv. conf.	[57.89, 70.75]	-	[0.0, 18.39]	[68.96, 75.24]	[69.49, 73.25]

Tabla 5. Estabilidad y precisión del modelo `phi3:mini`. Se muestra la precisión de 5 repeticiones de cada escenario junto con su promedio, desvío estándar e intervalo de confianza del 95%. Formato utilizado: RAW Completo.

Los estrechos intervalos de confianza del 95% demuestran estadísticamente que el rendimiento de `phi3:mini` es consistente. Al obtener rangos de variación tan acotados, se comprueba que el modelo no está adivinando ni sufriendo de fluctuaciones aleatorias graves, sino que mantiene una capacidad de clasificación estable y predecible al analizar los registros en cada nueva ejecución.

Modelos

Habiendo comprobado la estabilidad del modelo principal, se procedió a someter al modelo secundario (Llama3.2:3b) al mismo rigor de 5 iteraciones para evaluar el impacto de la arquitectura en la precisión del análisis.

Como se observa en la Tabla 6, aunque poseen una escala de parámetros equivalente, llama3.2:3b presentó resultados altamente erráticos en la detección de intentos de escalada de privilegios (Escenario 2), promediando una precisión pobre del 34.29% con una desviación estándar severa del 16.29%, perdiendo el contexto de seguridad repetidas veces.

	Escenario 1	Escenario 2	Escenario 3	Escenario 4	Escenario 5
Repetición 1	53.96%	28.57%	20.0%	68.42%	71.37%
Repetición 2	54.68%	57.14%	30.0%	72.37%	76.92%
Repetición 3	49.64%	28.57%	20.0%	65.79%	78.21%
Repetición 4	51.08%	42.86%	30.0%	64.47%	73.08%
Repetición 5	51.8%	14.29%	20.0%	61.84%	77.78%
Promedio	52.23%	34.29%	24.0%	66.58%	75.47%
Desvío Est.	2.07%	16.29%	5.48%	4.01%	3.06%
Interv. conf.	[49.65, 54.81]	[14.07, 54.51]	[17.2, 30.8]	[61.6, 71.56]	[71.67, 79.27]

Tabla 6. Estabilidad y precisión del modelo llama3.2:3b. Se muestra la precisión de 5 repeticiones de cada escenario junto con su promedio, desvío estándar e intervalo de confianza del 95%. Formato utilizado: RAW Completo.

Para confirmar si estas diferencias de rendimiento eran estadísticamente significativas o producto del azar, se aplicó una prueba t de Student comparando los promedios de ambos modelos (Tabla 7).

Los resultados del análisis ($p < 0.05$) confirman que las diferencias de rendimiento observadas entre los modelos son estadísticamente significativas y no producto del azar. El modelo *phi3:mini* demostró una superioridad notable en los escenarios 1, 2 y 4, su desempeño se destaca en el escenario 2, donde superó a su contraparte por un margen del 51,42%. En contraste, *llama3.2:3b* exhibió una ligera ventaja en los escenarios 3 y 5. Cabe señalar que, a pesar de esa leve ventaja, ambos modelos manifestaron severas dificultades al clasificar los registros del escenario 3, lo que evidencia una mayor complejidad de análisis en dichos logs. En términos generales, el modelo *phi3:mini* consolidó el rendimiento más robusto y eficiente de la comparativa.

Escenario	Promedio phi3:mini	Promedio llama3.2:3b	p valor	Conclusión (p < 0.05)
1	64.32%	52.23%	0.0041	phi3:mini es significativamente mejor en este escenario.
2	85.71%	34.29%	0.0021	phi3:mini es significativamente mejor en este escenario.
3	8.00%	24.00%	0.0092	llama3.2:3b es significativamente mejor en este escenario.
4	72.10%	66.58%	0.0364	phi3:mini es significativamente mejor en este escenario.
5	71.37%	75.47%	0.0372	llama3.2:3b es significativamente mejor en este escenario.

Tabla 7. Análisis de significancia estadística (Prueba t de Student) entre phi3:mini y llama3.2:3b. Se muestra el promedio de ambos modelos en cada escenario junto al p valor y la conclusión del análisis.

Sin embargo, como podemos observar en la Figura 4, *llama3.2:3b* es sustancialmente más rápido, promediando tiempos estables de inferencia por debajo de los 5 segundos por log, mientras que *phi3:mini* requiere entre 6 y 7 segundos. Esto marca una clara relación entre la precisión del análisis y el costo temporal de la inferencia.

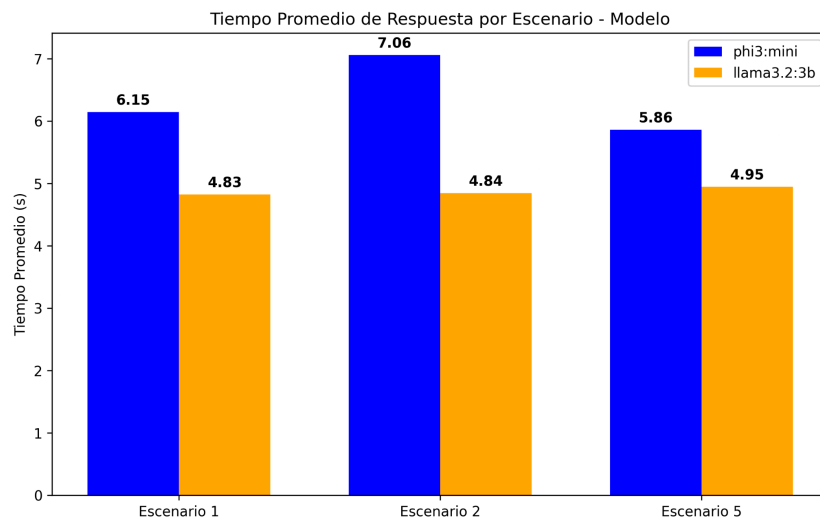


Figura 4. Tiempo de respuesta de los modelos. Se muestra el tiempo promedio de respuesta medidos en segundos para los escenarios 1, 2 y 5 de los modelos phi3:mini y llama3.2:3b. Formato utilizado: RAW completo.

Logs de ruido

La evaluación sobre el dataset de ruido (actividad normal) arrojó una precisión del 64.34%, identificando un total de 87 falsos positivos sobre 244 eventos evaluados. El análisis de estos fallos demuestra que las tareas cotidianas y legítimas del sistema operativo contienen cadenas sintácticas complejas que confunden al modelo compacto.

Si bien, al haber realizado esta prueba en el formato RAW reducido la ausencia de ciertos metadatos puede provocar que el LLM sobre-reaccione ante la aparición aislada de palabras sensibles o rutas del sistema clasificando eventos ordinarios e inofensivos bajo etiquetas de riesgo MEDIO o ALTO, se aproxima a la cantidad de falsos positivos detectados en los escenarios.

Eventos Totales	Aciertos	Precisión	Falsos Positivos
244	157	64.34%	87

Tabla 8. Precisión del análisis de los logs de ruido. Se muestra el total de eventos analizados junto a la precisión del modelo, y los falsos positivos. Modelo utilizado: phi3:mini. Formato utilizado: RAW reducido.

7.1 Comparación con los objetivos

Objetivo específico	Resultado obtenido	Estado
Contrastar los resultados de cada escenario del LLM contra el baseline en términos de precisión, coincidencia y tiempo promedio de respuesta.	Se demostró que la coincidencia de detección se concretó en 4 de los 5 escenarios, el LLM habiendo clasificado mal dos logs críticos y el baseline detectando en todos los escenarios. El LLM presentó un tiempo de respuesta promedio de 6.69 segundos.	Cumplido
Evaluar la influencia de la estructura de los datos en la capacidad de comprensión del modelo, comparando la tasa de precisión del LLM y el tiempo promedio de respuesta al procesar tres formatos distintos: RAW completo, RAW reducido y JSON reducido	Se evaluaron y contrastaron las tres estructuras de datos. Se demostró que la que mejores resultados ofrece es el formato RAW completo, mientras que el formato <i>JSON Reducido</i> , y en menor medida el RAW Reducido, degradan el razonamiento del modelo.	Cumplido
Evaluar y analizar la variación en el porcentaje de aciertos del modelo ante diferentes instrucciones y ejemplos provistos.	Se implementó y documentó la experimentación con tres variantes estructurales de prompts bajo la estrategia few-shot. Se demostró que a prompts más largos se diluyen las instrucciones e incrementan las alucinaciones semánticas.	Cumplido

Objetivo específico	Resultado obtenido	Estado
Evaluar el rendimiento de los modelos <i>phi3:mini</i> y <i>llama3.2:3b</i> en cuanto a la precisión y tiempo promedio de respuesta.	Se ejecutó el análisis comparativo entre los modelos bajo entornos idénticos, demostrando la superioridad del modelo <i>phi3:mini</i> para la interpretación de logs del kernel de Linux y la relación entre la precisión del análisis y el costo temporal de la inferencia ya que <i>llama3.2:3b</i> es más rápido.	Cumplido

7.2 Limitaciones identificadas

Se identificó una pérdida de la capacidad analítica del modelo al reducir los campos de los registros. Al intentar optimizar el consumo de tokens mediante el formato RAW Reducido, la eliminación agresiva de variables cegó el razonamiento del modelo. Esto provocó que en el escenario 2 (intentos de escalada de privilegios) la precisión cayera al 28.57%, demostrando que simplificar en exceso los metadatos de las *syscalls* impide al LLM comprender el contexto de seguridad del evento.

Además, se observó una inestabilidad y saturación de la ventana de contexto ante prompts extensos, los modelos compactos de menos de 4.000 millones de parámetros demostraron una alta sensibilidad al tamaño de las instrucciones. Al implementar el *Prompt 2* (cuatro ejemplos de referencia), el modelo sufrió un colapso cognitivo en los escenarios con mayor cantidad de eventos (Escenarios 1 y 5), reduciendo drásticamente su precisión. Esto evidencia la incapacidad del modelo para retener directivas lógicas si el prompt consume rápidamente sus tokens de atención disponibles.

La solución presenta una incapacidad de correlación temporal. Debido a la granularidad unitaria adoptada (análisis aislado de un log por prompt), no se pueden detectar ataques multi-paso. Además, esto se tradujo en una tasa de 87 falsos positivos en el dataset de ruido, ya que el modelo catalogó actividades rutinarias y sin peligro (*systemd*, *cron*, *sshd*) como riesgos elevados al evaluarlas de manera aislada y fuera de su contexto secuencial.

Por último, como el pipeline procesa los eventos registro por registro de manera secuencial, arrojando un tiempo promedio de respuesta de entre 5.86 y 7.96 segundos por cada log individual, en un entorno de producción real donde se generan cientos de líneas por minuto, esta velocidad de inferencia provocaría un cuello de botella crítico, haciendo inviable el uso del sistema actual para auditorías y respuestas tempranas en tiempo real.

7.3 Propuestas de mejora y trabajo futuro

Con el propósito de optimizar el rendimiento del pipeline desarrollado sin incrementar la complejidad de la infraestructura de hardware, se proponen las siguientes mejoras lógicas y prácticas para futuros desarrollos.

Diseñar un formato optimizado intermedio. En lugar de utilizar los dos extremos actuales (RAW Completo y RAW Reducido), se propone crear un formato que conserve más campos que fueron eliminados. De este modo, se reducirá el tamaño del texto sin cegar la capacidad de razonamiento del modelo en escenarios de escalada de privilegios.

Se propone optimizar la ingeniería de prompts utilizando solo dos ejemplos. Habiendo comprobado que cuatro ejemplos saturan la memoria del modelo y degradan la precisión en muestras grandes, se propone diseñar variantes de prompts optimizadas que utilicen estrictamente dos ejemplos. El trabajo se centraría en probar distintas combinaciones de ejemplos (distintos niveles de riesgos) y analizar sus resultados.

8. Conclusiones

El desarrollo de este trabajo ha permitido estructurar un pipeline de análisis de seguridad automatizado y local, abordando el problema en la administración de sistemas operativos respecto a la sobrecarga y la fatiga por alertas asociadas al análisis manual de registros de *auditd*. A través de la experimentación en un entorno con recursos de hardware limitados (GPU de 4GB de VRAM), la investigación aporta valor en las siguientes dimensiones.

En primer lugar se validó la viabilidad de implementar modelos de lenguaje de parámetros reducidos, como *phi3:mini*, demostrando mediante validación estadística de múltiples iteraciones que son capaces de analizar la sintaxis técnica de los registros y traducirlas de forma autónoma a clasificaciones de riesgo. Esto valida una alternativa viable para organizaciones que no pueden enviar sus datos internos a APIs comerciales en la nube. Así mismo, un aporte significativo radica en evidenciar que las técnicas de optimización sintáctica de datos (como la reducción agresiva de campos y utilización de JSON) no se traducen linealmente en mejoras para modelos compactos. Contrario a la intuición inicial, el modelo asimila de manera óptima la estructura cruda nativa separada por espacios (RAW Completo) generada por el kernel.

Al abordar de manera directa la pregunta de investigación planteada al inicio de este trabajo, orientada a determinar si el modelo *phi3:mini* utilizando *few-shot prompting* puede detectar eventos de seguridad en logs de *auditd* con mayor precisión que las reglas manuales de *ausearch* en un hardware con GPU de 4GB y sin conexión externa, la evidencia recolectada obliga a proponer una respuesta matizada. La respuesta definitiva a este interrogante es negativa en términos de precisión global absoluta y de eficiencia temporal, pero es afirmativa en lo que respecta a la especificidad analítica y a la mitigación de la fatiga operativa del administrador del sistema. Mientras que las reglas manuales del baseline implementadas mediante *ausearch* garantizan una tasa de detección del cien por ciento, carecen por completo de adaptabilidad contextual, lo que provoca que arrastren un volumen masivo de ruido operativo ordinario que satura al administrador de sistemas.

Por su parte, el modelo local *phi3:mini* demostró una precisión superior al 70% en la mayoría de los escenarios de ataque sobre el formato RAW Completo. No obstante, el LLM no puede considerarse más preciso de forma absoluta que el baseline debido a la aparición de falsos negativos críticos, tales como la incapacidad de decodificar argumentos en formato hexadecimal en el escenario 2 o el exceso de razonamiento semántico en el escenario 4, donde catalogó una modificación crítica mediante *visudo* como una tarea de mantenimiento ordinaria y esperada. Adicionalmente, el costo temporal de la inferencia del LLM introduce una limitación para entornos de producción masivos. Registrar un tiempo promedio de respuesta de 6.69 segundos por cada log evaluado genera un cuello de botella frente a los milisegundos casi instantáneos que requiere el filtrado sintáctico de *ausearch*, imposibilitando el uso del estado actual del pipeline para auditorías y respuestas tempranas en tiempo real en empresas medianas. El verdadero valor del modelo no radica en reemplazar el filtrado tradicional, sino en actuar como una valiosa segunda capa de análisis semántico capaz de reducir drásticamente el tiempo que un operador humano gasta interpretando cadenas RAW complejas.

Más allá de la respuesta a la pregunta principal, la experimentación arrojó lecciones complementarias cruciales para la utilización de LLMs. Se identificó una clara reducción del contexto dependiendo del formato de los datasets, observando que formatos como el JSON Reducido desplomaron las precisiones a rangos deficientes

de entre el 34% y el 35% debido a que la introducción de caracteres de control estructurados satura la ventana de atención del modelo compacto. El hardware termina consumiendo tokens esenciales en decodificar la estructura gramatical del string en lugar de razonar sobre las implicaciones de ciberseguridad del log, confirmando la alta sensibilidad de estos modelos al formato de entrada. Del mismo modo, se comprobó que la ingeniería de prompts en modelos locales reducidas se beneficia de instrucciones más cortas pero bien estructuradas, puesto que la inyección de instrucciones excesivamente largas con múltiples ejemplos, como el Prompt 2, diluye las directivas primarias e incrementa significativamente las alucinaciones semánticas en muestras de gran tamaño, siendo el Prompt 1 de solo dos ejemplos la opción más estable.

Finalmente, la adopción de una granularidad unitaria orientada al procesamiento log por log impidió al pipeline la capacidad de correlacionar eventos a lo largo del tiempo, anulando la posibilidad de detectar ataques multi-paso complejos. Esta desconexión secuencial explica la tasa de 87 falsos positivos identificada en el dataset de ruido, donde el modelo catalogó actividades rutinarias y seguras como riesgos elevados al evaluarlas de forma completamente aislada de su flujo temporal. En conclusión, la solución ideal para la auditoría moderna de sistemas operativos no consiste en la sustitución de las herramientas nativas por modelos de lenguaje, sino en un enfoque híbrido donde *ausearch* actúe como un primer filtro masivo e instantáneo, derivando únicamente aquellos registros anómalos o de difícil interpretación hacia el LLM local para su categorización y resumen asistido, resguardando en todo momento la privacidad de la infraestructura.

9. Referencias bibliográficas

Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). Small Language Models are the Future of Agentic AI. <https://doi.org/10.48550/arXiv.2506.02153>

Bui, M. T., Boffa, M., Valentim, R. V., Navarro, J. M., Chen, F., Bao, X., Ben Houidi, Z., & Rossi, D. (2024). A systematic comparison of large language models performance for intrusion detection. *Proceedings of the ACM on Networking*, 2(CoNEXT4), 1–23. <https://doi.org/10.1145/3696379>

Cadet, E., Etim, E. D., Essien, I. A., Erigha, E. D., Babatunde, L. A., Ajayi, J. O., & Obuse, E. (2024). Large Language Models for Cybersecurity Policy Compliance and Risk Mitigation. *International Journal of Scientific Research in Humanities and Social Sciences*, 1(2), 612–643. <https://doi.org/10.32628/IJSRSSH242560>

Das, N., Amini, M. H., & Wu, Y. (2024). Security and privacy challenges of large language models: A survey. <https://doi.org/10.48550/arXiv.2402.00888>

Gadesha, Vrunda. What is few shot prompting? <https://www.ibm.com/think/topics/few-shot-prompting>

Karlsen, E., Luo, C., Zincir-Heywood, A. N., & Heywood, M. I. (2023). Benchmarking large language models for log analysis, security, and interpretation. <https://doi.org/10.48550/arXiv.2311.14519>

Kheddar, H. (2024). Transformers and large language models for efficient intrusion detection systems: A comprehensive survey. *World Journal of Advanced Research and Reviews*, 24(3), 2329–2341. <https://doi.org/10.48550/arXiv.2408.07583>

Ma, L., Yang, W., Li, Y., Fei, B., Zhou, M., Li, B., Jiang, S., Xu, B., & Xiao, Y. (2026). AdaptiveLog: An adaptive log analysis framework with the collaboration of large and small language model. *ACM Transactions on Software Engineering and Methodology*, 35(5), 1–42. <https://doi.org/10.1145/3749840>

Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2024). Large Language Models: A Survey. <https://doi.org/10.48550/arXiv.2402.06196>

Tejero-Fernández, J. J., & Sánchez-Macián, A. (2025). Evaluating language models for threat detection in IoT security logs. <https://doi.org/10.48550/arXiv.2507.02390>

Varughese, Jobit. What is in-context learning? <https://www.ibm.com/think/topics/in-context-learning>

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., ... & Wen, J. R. (2023). A survey of large language models. <https://doi.org/10.48550/arXiv.2303.18223>

Xu, M., Sharma, V., Pandey, S., Srinivasan, S. M., & Blackburn, J. (2024). Large language models for cyber security: A systematic literature review. <https://doi.org/10.48550/arXiv.2405.04760>

10. Anexos

Anexo A — Prompts

Prompt 1:

Sos un experto en ciberseguridad analizando logs de auditd de Linux.

Tu tarea es analizar el siguiente log y responder ÚNICAMENTE en formato JSON con la siguiente estructura:

```
{"riesgo": "ALTO/MEDIO/BAJO/INFO", "justificacion": "Explicación en una línea"}
```

REGLA Estricta: NUNCA uses comillas dobles (") dentro del texto de la justificación. Si necesitas citar algo, usa comillas simples (').

Ejemplo 1:

```
Log: type=SYSCALL msg=audit(1621234567.890:123): arch=c000003e syscall=59
success=yes exit=0 a0=55f1a2b3c4d5 a1=55f1a2b3c4e0 a2=55f1a2b3c4f0 a3=8 items=2
ppid=1234 pid=5678 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0
tty=pts0 ses=1 comm="whoami" exe="/usr/bin/whoami" key="root_commands"
```

Respuesta: {"riesgo": "BAJO", "justificacion": "El usuario ejecutó whoami con privilegios, lo cual es normal en tareas administrativas, pero requiere monitoreo."}

Ejemplo 2:

```
Log: type=SYSCALL msg=audit(1700000000.001:101): arch=c000003e syscall=257
success=no exit=-13 a0=ffffff9c a1=7ffdc... items=1 ppid=2000 pid=2002 auid=1000 uid=1000
gid=1000 euid=1000 tty=pts0 comm="cat" exe="/usr/bin/cat" key="shadow_access"
```

Respuesta: {"riesgo": "ALTO", "justificacion": "Intento fallido de lectura de un archivo crítico (/etc/shadow) por falta de permisos (exit=-13), posible intento de robo de credenciales."}

Log a analizar:

```
{log_line}
```

Prompt 2:

Sos un experto en ciberseguridad analizando logs de auditd de Linux.

Tu tarea es analizar el siguiente log y responder ÚNICAMENTE en formato JSON con la siguiente estructura:

```
{"riesgo": "ALTO/MEDIO/BAJO/INFO", "justificacion": "Explicación en una línea"}
```

REGLA Estricta: NUNCA uses comillas dobles (") dentro del texto de la justificación. Si necesitas citar algo, usa comillas simples (').

Ejemplo 1:

```
Log: type=SYSCALL msg=audit(1700000010.000:105): arch=c000003e syscall=257
success=yes exit=3 a0=ffffff9c a1=7ffdc... items=1 ppid=1 pid=500 auid=unset uid=0 euid=0
tty=(none) comm="systemd" exe="/lib/systemd/systemd"
```

Respuesta: {"riesgo": "INFO", "justificacion": "Proceso interno del sistema operativo ejecutando tareas de rutina sin impacto de seguridad."}

Ejemplo 2:

```
Log: type=SYSCALL msg=audit(1621234567.890:123): arch=c000003e syscall=59
success=yes exit=0 ... uid=1000 euid=1000 comm="cat" exe="/usr/bin/cat"
key="passwd_changes"
```

Respuesta: {"riesgo": "BAJO", "justificacion": "El usuario realizó una lectura normal de un archivo del sistema, comportamiento esperado y sin privilegios elevados."}

Ejemplo 3:

```
Log: type=USER_AUTH msg=audit(1700000020.000:110): pid=1000 uid=1000 auid=1000
ses=1 msg='op=PAM:authentication grantors=? acct="root" exe="/usr/bin/su" hostname=?
addr=? terminal=pts/0 res=failed'
```

Respuesta: {"riesgo": "MEDIO", "justificacion": "Intento de autenticación fallido para escalar privilegios a root mediante su, requiere revisión."}

Ejemplo 4:

Log: type=SYSCALL msg=audit(1700000000.001:101): arch=c000003e syscall=257
 success=no exit=-13 ... uid=1000 euid=1000 comm="cat" exe="/usr/bin/cat"
 key="shadow_access"
 Respuesta: {"riesgo": "ALTO", "justificacion": "Intento fallido de lectura de un archivo crítico (/etc/shadow) por falta de permisos (exit=-13), posible intento de robo de credenciales."}
 Log a analizar:
 {log_line}

Prompt 3:

Sos un experto en ciberseguridad analizando logs de auditd de Linux.
 Tu tarea es analizar el siguiente log y responder ÚNICAMENTE en formato JSON con la siguiente estructura:
 {"riesgo": "ALTO/MEDIO/BAJO/INFO", "justificacion": "Explicación en una línea"}
 CRITERIOS DE CLASIFICACIÓN:
 - INFO: Ruido del sistema, demonios (systemd, cron) o autenticaciones normales.
 - BAJO: Lectura exitosa de archivos generales por usuarios sin privilegios.
 - MEDIO: Uso de comandos administrativos (sudo, su), modificaciones menores o intentos de autenticación fallidos.
 - ALTO: Modificación de archivos críticos (/etc/sudoers), acceso denegado (exit=-13) a credenciales (/etc/shadow), o comandos de persistencia (useradd).
 REGLA Estricta: NUNCA uses comillas dobles (") dentro del texto de la justificación. Si necesitas citar algo, usa comillas simples (').
 Ejemplo 1:
 Log: type=SYSCALL msg=audit(1700000010.000:105): comm="systemd" uid=0
 Respuesta: {"riesgo": "INFO", "justificacion": "Actividad de demonio del sistema sin relevancia de seguridad."}
 Ejemplo 2:
 Log: type=USER_AUTH msg=audit(1700000020.000:110): comm="su" res=failed
 Respuesta: {"riesgo": "MEDIO", "justificacion": "Intento fallido de cambio de usuario, posible intento de escalada."}
 Ejemplo 3:
 Log: type=SYSCALL msg=audit(1700000000.001:101): comm="cat" key="shadow_access" exit=-13
 Respuesta: {"riesgo": "ALTO", "justificacion": "Acceso denegado a credenciales críticas, fuerte indicador de compromiso."}
 Log a analizar:
 {log_line}

Anexo B — Código de la solución

El código de la solución se encuentra en el siguiente repositorio de github:
<https://github.com/tomysturtz/tp-final-sor2>